Minitrabalho DAA – "Análise de Redes Ficcionais" (Versão 1)

Introdução

A análise de redes é um dos temas mais estudados em teoria dos grafos e pode abranger uma gama extremamente ampla de tipos de interações, desde relações sociais, infraestruturas, sistemas biológicos ou, no caso deste minitrabalho, relações entre personagens de universos ficcionais, como o da saga de *Game of Thrones* (*A Song of Ice and Fire*) e o dos super-heróis da Marvel Comics.

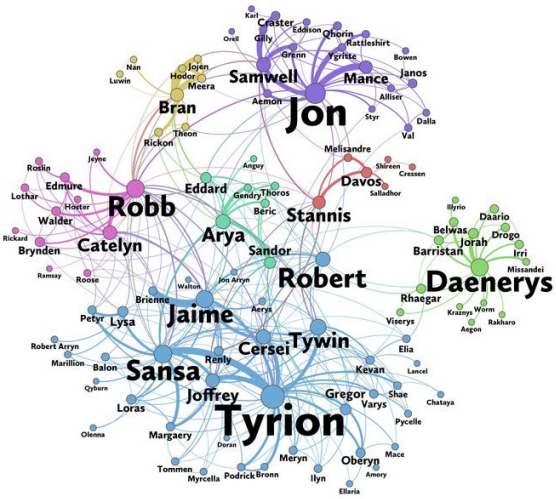

Figura 1: Exemplo de rede de personagens do livro 1 de *Game of Thrones*. *Fonte:* https://www.kaggle.com/code/mmmarchetti/game-of-thrones-network-analysis

Nos últimos anos, a análise de redes evoluiu da visualização de pequenos grafos (< 100 vértices) para a análise estatística em larga escala ($\approx$ milhões de vértices). A disponibilidade de recursos computacionais levou à definição de várias métricas estatísticas para descrever e comparar redes (focando-nos, neste caso, principalmente em grafos não orientados), por exemplo:

- Distribuição de graus (*quão densa é a rede?*);
- Diâmetro e comprimento médio do caminho (*quão bem conectada é a rede?*);
- Centralidade (*quais são os vértices centrais ou importantes?*).

O objetivo deste trabalho é implementar em Python um conjuto de métricas estatísticas para analisar universos ficcionais. Estes algoritmos estão organizados numa API (`CentralityAnalyzer`). Um exemplo de análise deste tipo é realizado pela ferramenta **Network Navigator**[1]. Para contextualização e orientação de implementação das métricas, consulte o **documento de apoio** disponibilizado juntamente com este enunciado no Moodle.

Datasets

São fornecidos quatro *datasets* em formato CSV com colunas `Source, Target, weight`, onde o peso representa o número de co-ocorrências entre dois personagens:

- **got_book1.csv (Pequeno)**: Rede de co-ocorrências do livro 1 de *A Song of Ice and Fire*[2] com 187 vértices e 684 arestas.

[1] https://networknavigator.jrladd.com/about.html
[2] https://github.com/mathbeveridge/asoiaf

- **got_full.csv (Médio)**: Rede de co-ocorrências da saga completa de *A Song of Ice and Fire*[2] com 796 vértices e 2823 arestas (interações entre personagens ao longo dos 5 livros).
- **marvel_small.csv (Médio)**: Subgrafo compacto do universo Marvel[3] com 327 vértices e 9891 arestas.
- **marvel_full.csv (Grande)**: Grafo completo do universo Marvel[4,5] com 6421 vértices e 167112 arestas.

Requisitos do trabalho

O grupo deverá criar o código pedido e responder às questões num **Jupyter Notebook**, submetido no Moodle conforme indicado na Secção Submissão.

Regras fundamentais:

- A estrutura do relatório deve seguir a mesma estrutura de secções apresentada abaixo.
- Não é permitida a utilização de módulos Python para representação de grafos ou para o cálculo das métricas de centralidade (e.g., `networkx`, `igraph`). Deverá implementar a sua própria representação de grafos, ou usar como base a classe `Graph` fornecida nos exercícios da Semana 7.
- A utilização de ferramentas de inteligência artificial (IA) não é expressamente proibida, mas todas as explicações, conclusões e análises **deverão ser apresentadas com as suas próprias palavras**. O uso indiscriminado de IA para gerar explicações ou análises é desencorajado e penalizado caso não seja demonstrado compreensão genuína do trabalho.

1. API `CentralityAnalyzer`

Implemente a classe `CentralityAnalyzer` e cada um dos seus métodos listados na Tabela 1, respeitando a sua assinatura.

1.1. Análise de complexidade

Para cada método implementado, apresente uma análise da complexidade temporal em termos de n (número de vértices) e m (número de arestas), justificando cada passo antes de concluir sobre a complexidade total.

1.2. [1,0 valor] Construtor e estrutura de dados utilizada

O construtor da classe recebe um objeto `graph` instanciado a partir de um ficheiro CSV, no formato anteriormente descrito (ver Secção Datasets), e inicializa a estrutura de dados interna.

Apresente uma estimativa teórica do espaço de memória total utilizado pela sua aplicação (pelos seus atributos) em termos do número de vértices e arestas do grafo analisado.

2. Análise Estrutural dos Grafos

Implemente os métodos e responda às questões apresentadas nas subsecções de 2.1 a 2.4. Apresente uma validação dos métodos implementados, de modo a verificar a sua correção e a analisar os resultados.

[3] https://github.com/melaniewalsh/sample-social-network-datasets
[4] https://www.kaggle.com/datasets/csanhueza/the-marvel-universe-social-network
[5] http://syntagmatic.github.io/exposedata/marvel/

Tabela 1: API `CentralityAnalyzer`

Método	Descrição
`__init__(graph)`	Recebe o grafo a ser analisado em `graph`. Inicializa os atributos da classe.
`bfs(source)`	Efetua uma travessia em largura no grafo a partir de um vértice `source`.
`num_components()`	Devolve o número de componentes conexas do grafo analisado.
`largest_component()`	Devolve um subgrafo (tipo `Graph`) com os vértices e arestas da maior componente conexa do grafo analisado.
`degree_distribution()`	Devolve um `dict` {`grau` → `número de vértices`} com a distribuição de graus do grafo.
`diameter()`	Diâmetro da maior componente conexa (maior distância entre qualquer par de vértices).
`degree_centrality()`	Devolve um `dict` {`vertexid` → `score`} em que o valor da métrica de centralidade correspondente para o vértice é armazenado em `score`.
`closeness_centrality()`	Devolve um `dict` {`vertexid` → `score`} em que o valor da métrica de centralidade correspondente para o vértice é armazenado em `score`.
`eigenvector_centrality()`	Devolve um `dict` {`vertexid` → `score`} em que o valor da métrica de centralidade correspondente para o vértice é armazenado em `score`. Devolve também o número k de iterações realizadas.
`betweenness_centrality()`	Devolve um `dict` {`vertexid` → `score`} em que o valor da métrica de centralidade correspondente para o vértice é armazenado em `score`.

2.1. [1,0 valores] BFS

Implemente o método `bfs(source)` que efetua uma travessia em largura no grafo a partir do vértice fonte `source`. Pode ter por base a API **BfsPaths** desenvolvida nos exercícios da Semana 7 (ver Moodle). Este método será reutilizado por vários outros métodos da classe, por isso, deve implementá-lo com cuidado. Por esta mesma razão, tem a liberdade de modificar a assinatura do método conforme sentir necessidade.

Descreva o funcionamento do algoritmo BFS, indicando a estrutura de dados auxiliar utilizada e o seu papel para o algoritmo. Analise a complexidade temporal do método, tendo em conta a representação de grafo utilizada.

2.2. [1,5 valores] Conectividade

Implemente os métodos de conectividade `num_components()` e `largest_component()`, utilizando por base a classe `ConnectedComponents` implementada nos exercícios da Semana 8. Analise a complexidade temporal dos métodos.

Aplique-os aos *datasets* **got_book1.csv** e **got_full.csv** e responda às seguintes questões:

- O grafo é conexo? Quantas componentes conexas existem?
- Qual o tamanho (ordem) da maior componente? Os vértices fora da maior componente têm alguma interpretação no contexto narrativo de cada universo?
- Verifique se dois personagens específicos à sua escolha estão conectados e apresente o

caminho (sequência de vértices) entre eles.
- Compare a estrutura de conectividade nos dois datasets. A rede torna-se mais ou menos conexa ao longo dos livros da saga?

2.3. [1,5 valores] Distribuição de graus e top-10

Implemente o método `degree_distribution()`. Analise a complexidade temporal e espacial deste método.

Construa o histograma da distribuição de graus para os *datasets* **got_full.csv** e **marvel_full.csv**. Com base na distribuição de graus, responda:

- A distribuição segue uma **lei de potência**, também chamada de distribuição de "cauda longa" (poucos vértices com graus muito altos e muitos vértices com graus mais baixos, apresentando uma longa cauda que se estende para valores de graus mais baixos)? Comente o que esta forma revela sobre a estrutura da rede.
- Existe alguma relação entre as conclusões nos dois universos ficcionais? Existem diferenças entre a distribuição dos dois universos? O que isto pode significar?

Adicionalmente, apresente o top-10 de personagens por grau para os *datasets* **got_book1.csv** e **got_full.csv** e responda:

- Os personagens mais conectados no livro 1 mantêm a sua posição na saga completa? Comente as diferenças. Guarde estes resultados pois servirão de base de comparação para as conclusões que terá de apresentar na Secção 3.

2.4. [1,5 valores] Diâmetro

Implemente o método `diameter()`, que calcula o diâmetro da maior componente conexa do grafo. O diâmetro corresponde ao maior valor de distância, $d(u, v)$, entre todos os pares de vértices da componente conexa. Consulte o **documento de apoio** disponibilizado no Moodle para mais detalhes sobre esta métrica.

Aplique-o aos *datasets* **got_full.csv** e **marvel_full.csv** e responda:

- Qual o diâmetro de cada *dataset*? Apresente um par de personagens que realize essa distância máxima e a sequência de vértices do caminho que liga este par de vértices.
- Interprete os valores obtidos à luz da teoria dos *"seis graus de separação"*[6].

3. Métricas de Centralidade

Implemente os métodos e responda às questões apresentadas nas subsecções de 3.1 a 3.4. Apresente uma validação dos métodos implementados, de modo a verificar a sua correção e a analisar os resultados.

3.1. [1,0 valor] Degree Centrality (DC)

A métrica DC simplesmente normaliza o grau de cada vértice pelo número máximo possível de vértices vizinhos. Isto é, divide o grau do vértice por $(n - 1)$. É a métrica mais simples e serve de base para comparar com as métricas seguintes (ver mais detalhes no **documento de apoio**).

Implemente o método `degree_centrality()` e analise a sua complexidade temporal e espacial.

Apresente o top-10 de DC para os *datasets* **got_book1.csv** e **got_full.csv** e responda:

[6] https://en.wikipedia.org/wiki/Six_degrees_of_separation

- Compare o top-10 de DC com o top-10 por grau da Secção 2.3. São iguais? Porquê?
- Os personagens mais centrais do livro 1 são os mesmos da saga completa?

3.2. [2,5 valores] Closeness Centrality (CC)

A métrica CC mede a proximidade de um vértice a todos os outros: um vértice com CC elevada consegue "alcançar" todos os outros em poucas ligações (i.e., número de arestas). A fórmula a ser adotada é a de Wasserman-Faust, por ser a mais adequada para grafos desconexos (ver mais detalhes no **documento de apoio**).

Implemente o método `closeness_centrality()` e analise a sua complexidade temporal e espacial.

Valide a implementação: calcule manualmente a CC de um vértice num grafo pequeno construído à mão e verifique que a implementação reproduz o resultado esperado.

Apresente o top-10 de DC para o dataset **got_book1.csv** e responda:

- O personagem **Eddard-Stark** surge no topo do ranking?
- Remova **Eddard-Stark** do grafo e recalcule a CC de todos os personagens. Identifique personagens cuja CC mais diminui em valor absoluto e apresente a variação numérica ($\Delta CC = CC_{\text{antes}} - CC_{\text{depois}}$). O que pode isto significar? Com base no papel narrativo desses personagens no livro 1, explique por que dependem tanto de Eddard-Stark como intermediário.

3.3. [2,5 valores] Eigenvector Centrality (EC)

A métrica EC atribui a cada vértice um score proporcional à soma dos scores dos seus vizinhos: um vértice é importante se os seus vizinhos também são importantes. Por este algoritmo, o score de cada vértice converge para o vetor próprio dominante da matriz de adjacência do grafo, calculado iterativamente pelo método *Power Iteration* (ver mais detalhes no **documento de apoio**).

Implemente o método `eigenvector_centrality()` e analise a sua complexidade temporal e espacial. Para isso, identifique o custo de cada iteração em função de n e m, o número de iterações k realizadas até convergência, e o trabalho de normalização por iteração.

Valide a implementação: calcule manualmente a EC de um vértice num grafo pequeno construído à mão e verifique que a implementação reproduz o resultado esperado.

Apresente o top-10 de EC para o *dataset* **got_book1.csv** e responda:

- Compare com o top-10 de DC da Secção 3.1. Identifique um personagem que esteja no top-10 de EC mas não no top-10 de DC (ou vice-versa). Alternativamente, identifique um personagem que sobe ou desce no ranking EC vs. DC. O que pode isto significar? Com base no papel desse personagem na história do livro 1, tente explicar uma razão para este resultado.

3.4. [4,0 valores] Betweenness Centrality (BC)

A métrica BC mede o poder de intermediação de um vértices, ou seja, com que frequência aparece nos caminhos mais curtos entre todos os pares de vértices.

Implemente `betweenness_centrality()` utilizando algoritmo de Brandes (ver **documento de apoio**) e analise a sua complexidade temporal e espacial.

Valide a sua implementação: calcule manualmente a BC de um vértice num grafo pequeno construído à mão e verifique que a implementação reproduz o resultado esperado.

Apresente o top-10 de BC para o *dataset* **got_full.csv** e responda:

- Compare com o top-10 de CC (Secção 3.1) para o mesmo *dataset*. Identifique um personagem que esteja no top-10 de EC mas não no top-10 de CC (ou vice-versa). Alternativamente, identifique um personagem que sobe ou desce consideravelmente no ranking BC vs. CC. Com base no papel do personagem ao longo da saga, explique as diferenças.

Apresente o top-10 de BC para o *dataset* **got_book1.csv** e responda:

- Os personagens com maior poder de intermediação no livro 1 mantêm esse papel ao longo de toda a saga?

4. Análise empírica do tempo de execução

4.1. [1,0 valores] Escalabilidade dos algoritmos

Meça o tempo de execução de DC, EC, CC e BC nos quatro *datasets* `got_book1`, `got_full`, `marvel_small` e `marvel_full` e apresente um gráfico log-log com os resultados (eixo x: $(n+m)$; eixo y: segundos; uma curva por algoritmo).

Para obter medições mais estáveis, repita cada medição **pelo menos cinco vezes** nos três datasets mais pequenos (`got_book1`, `got_full` e `marvel_small`) e utilize a **mediana** dos tempos obtidos. Para o dataset `marvel_full`, dado o seu tamanho, é suficiente realizar apenas uma medição (caso demore demasiado tempo).

Responda às seguintes questões:

- A ordenação dos tempos T_{DC}, T_{EC}, T_{CC} e T_{BC} é consistente com as complexidades calculadas na Secção 3?
- As retas no gráfico log-log têm declives diferentes para cada algoritmo? O que isso revela sobre a ordem de complexidade?

4.2. [1,0 valores] Comparação empírica CC vs. BC

Com os tempos medidos na Secção 4.1, calcule a razão T_{BC}/T_{CC} para cada *dataset* e apresente um gráfico de barras (eixo x: *dataset*; eixo y: razão T_{BC}/T_{CC}). Responda:

- A razão é aproximadamente constante entre datasets? O que isso implica sobre as constantes escondidas na notação $O(\cdot)$ de cada algoritmo?
- Com base na implementação que fez, identifique a(s) operação(ões) que BC executa e CC não executa que explique a diferença empírica.

4.3. [1,0 valores] Verificação $T_{CC}/T_{EC} \approx n/k$

Com os tempos medidos na Secção 4.1, para cada dataset, compute a razão T_{CC}/T_{EC} e o valor n/k, onde k é o número de iterações reportado por `eigenvector_centrality`. Apresente um **gráfico de dispersão** com n/k no eixo x e T_{CC}/T_{EC} no eixo y, adicionando a reta $y = x$ como referência teórica.

Responda às seguintes questões:

- O declive observado pelos pontos da dispersão é próximo de 1? O que isso confirma sobre a relação teórica $T_{CC}/T_{EC} \approx n/k$?
- Os pontos aproximam-se da reta $y = x$, ou estão sistematicamente acima ou abaixo? Justifique o desvio com base nas constantes escondidas de cada algoritmo.
- O valor de k é semelhante entre os datasets? O que isso revela sobre a velocidade de convergência do algoritmo *Power Iteration* nestes grafos?

5. [0,5 valor] Questões Éticas

Tente resolver os problemas apenas com os integrantes do seu grupo antes de colaborar com outros. Escreva as suas respostas com as suas próprias palavras. Nunca deve partilhar o ficheiro fonte com as suas soluções com integrantes de outros grupos.

5.1. Se colaborou com alguém fora do seu grupo, indique aqui os respetivos nomes.

5.2. Deve citar todas as fontes que utilizou fora do material da UC.

Qualquer indício de plágio implica, automaticamente, na reprovação na avaliação periódica. Consoante a gravidade dos indícios de plágio, estes serão apropriadamente investigados e, caso se confirme que um estudante cometeu plágio, este estará automaticamente reprovado à UC.

Submissão

O prazo para entrega deste trabalho é o final do dia **11 de maio de 2025**. A entrega deve ser feita através do Moodle.

Deverá cumprir escrupulosamente as seguintes regras:

- Deve entregar o relatório num **Jupyter Notebook** com a mesma estrutura apresentada na secção "Requisitos do trabalho". O ficheiro `.ipynb` deve conter o histórico da execução, incluindo os *outputs* gerados.
- Deve garantir que é possível executar o notebook sem erros antes de submeter.
- Deve submeter o ficheiro `.ipynb` ou um ficheiro `.zip` que inclua o notebook e outros ficheiros que forem necessários para a sua execução. Não serão aceites outros formatos.

Recomendamos vivamente que as expressões matemáticas sejam apresentadas numa célula Markdown utilizando a sintaxe LaTeX para equações, como mostrado no exemplo abaixo. Alternativamente, podem inseri-las como uma imagem.

```
$\sum_{i=1}^{n} 1 = n$
```

$$\sum_{i=1}^{n} 1 = n$$

Nota: Os trabalhos que não cumprirem as regras acima só serão tratados e corrigidos após os integrantes do grupo estarem aprovados nas outras componentes de avaliação periódica (e, caso não fiquem aprovados, os trabalhos nem sequer serão vistos).

Tabela 2: Resumo das cotações.

Secção	**Componente**	**Cotação (val.)**
1.1	—	—
1.2	`__init__(graph)` + análise espacial	1,0
2.1	`bfs()` + estrutura auxiliar + complexidade	1,0
2.2	Métodos de conectividade + conclusões	1,5
2.3	`degree_distribution()` + histograma + top-10	1,5
2.4	`diameter()` + conclusões	1,5
3.1	`degree_centrality()` + complexidade + conclusões	1,0
3.2	`closeness_centrality()` + complexidade + conclusões	2,5
3.3	`eigenvector_centrality()` + complexidade + conclusões	2,5
3.4	`betweenness_centrality()` + complexidade + conclusões	4,0
4.1	Gráfico log-log de escalabilidade	1,0
4.2	Gráfico de barras T_{BC}/T_{CC} + análise da constante	1,0
4.3	Gráfico de dispersão T_{CC}/T_{EC} vs. n/k	1,0
5	Questões éticas	0,5
Total		**20,0**